

```

1 # %% [markdown]
2 # # 텍스트 요약 프로젝트 (Text Summarization)
3 # - **Abstractive**: seq2seq (3-stacked LSTM Encoder + LSTM Decoder) + Bahdanau Attention
4 # - **Extractive**: TextRank (summa)
5 # - **Dataset**: Amazon Fine Food Reviews (Reviews.csv)
6 #
7 # 루브릭 매핑
8 # 1. 전처리(분석→정제→정규화/불용어→분리→인코딩) ..... Cell 2~7
9 # 2. 학습 확인(loss 그래프 + 핵심단어 포함) ..... Cell 8~11
10 # 3. Extractive vs Abstractive 비교(표) ..... Cell 12~13
11
12 # %%
13 # =====
14 # Cell 0. 라이브러리
15 # =====
16 import numpy as np
17 import pandas as pd
18 import re
19 import matplotlib.pyplot as plt
20 from bs4 import BeautifulSoup
21
22 import nltk
23 from nltk.corpus import stopwords
24 nltk.download('stopwords')
25
26 from sklearn.model_selection import train_test_split
27 from tensorflow.keras.preprocessing.text import Tokenizer
28 from tensorflow.keras.preprocessing.sequence import pad_sequences
29 from tensorflow.keras.layers import Input, LSTM, Embedding, Dense, Concatenate
30 from tensorflow.keras.models import Model
31 from tensorflow.keras.callbacks import EarlyStopping
32 from tensorflow.keras.layers import Layer
33 from tensorflow.keras import backend as K
34 import tensorflow as tf
35
36 import warnings
37 warnings.filterwarnings('ignore')
38
39 stop_words = set(stopwords.words('english'))
40
41
42 # %%
43 # =====
44 # Bahdanau Attention Layer
45 # - 출처: thushv89/attention_keras (Aiffel 노트에서 사용하는 그 레이어)
46 # - 별도 attention.py 로 분리해도 됨. TF 2.x 기준.
47 # =====
48 class AttentionLayer(Layer):
49     """Bahdanau attention (https://arxiv.org/pdf/1409.0473.pdf)."""
50     def __init__(self, **kwargs):
51         super(AttentionLayer, self).__init__(**kwargs)
52
53     def build(self, input_shape):
54         assert isinstance(input_shape, list)
55         self.W_a = self.add_weight(name='W_a',
56                                     shape=tf.TensorShape((input_shape[0][2],
57                                                             input_shape[0][2])),
58                                     initializer='uniform', trainable=True)
59         self.U_a = self.add_weight(name='U_a',
60                                     shape=tf.TensorShape((input_shape[1][2],
61                                                             input_shape[0][2])),
62                                     initializer='uniform', trainable=True)
63         self.V_a = self.add_weight(name='V_a',
64                                     shape=tf.TensorShape((input_shape[0][2], 1)),
65                                     initializer='uniform', trainable=True)
66         super(AttentionLayer, self).build(input_shape)

```

```

66     def call(self, inputs, verbose=False):
67         assert type(inputs) == list
68         encoder_out_seq, decoder_out_seq = inputs
69
70         def energy_step(inputs, states):
71             assert isinstance(states, (list, tuple)), "States must be an iterable."
72             W_a_dot_s = K.dot(encoder_out_seq, self.W_a)
73             U_a_dot_h = K.expand_dims(K.dot(inputs, self.U_a), 1)
74             reshaped_Ws_plus_Uh = K.tanh(W_a_dot_s + U_a_dot_h)
75             e_i = K.squeeze(K.dot(reshaped_Ws_plus_Uh, self.V_a), axis=-1)
76             e_i = K.softmax(e_i)
77             return e_i, [e_i]
78
79         def context_step(inputs, states):
80             c_i = K.sum(encoder_out_seq * K.expand_dims(inputs, -1), axis=1)
81             return c_i, [c_i]
82
83         def create_initial_state(inputs, hidden_size):
84             fake_state = K.zeros_like(inputs)
85             fake_state = K.sum(fake_state, axis=[1, 2])
86             fake_state = K.expand_dims(fake_state)
87             fake_state = K.tile(fake_state, [1, hidden_size])
88             return fake_state
89
90         fake_state_c = create_initial_state(encoder_out_seq, encoder_out_seq.shape[-1])
91         fake_state_e = create_initial_state(encoder_out_seq, encoder_out_seq.shape[1])
92
93         last_out, e_outputs, _ = K.rnn(energy_step, decoder_out_seq, [fake_state_e])
94         last_out, c_outputs, _ = K.rnn(context_step, e_outputs, [fake_state_c])
95         return c_outputs, e_outputs
96
97     def compute_output_shape(self, input_shape):
98         return [
99             tf.TensorShape((input_shape[1][0], input_shape[1][1], input_shape[1][2])),
100             tf.TensorShape((input_shape[1][0], input_shape[1][1], input_shape[0][1]))
101         ]
102
103
104 # %%
105 # =====
106 # Cell 1. (전처리 1단계 - 분석) 데이터 로드 & 둘러보기
107 # =====
108 # Kaggle: https://www.kaggle.com/datasets/snap/amazon-fine-food-reviews
109 data = pd.read_csv("Reviews.csv", nrows=100000) # 메모리 고려해 10만행만 사용
110 data = data[['Text', 'Summary']]
111 print("원본 샘플 수:", len(data))
112
113 # 중복/결측 제거
114 data.drop_duplicates(subset=['Text'], inplace=True)
115 data.dropna(axis=0, inplace=True)
116 print("중복/결측 제거 후:", len(data))
117 data.head()
118
119
120 # %%
121 # =====
122 # Cell 2. (전처리 2단계 - 정제 & 3단계 - 정규화/불용어 제거)
123 # =====
124 # 약어(contraction) 사전
125 contractions = {
126     "ain't": "is not", "aren't": "are not", "can't": "cannot", "could've": "could have",
127     "couldn't": "could not", "didn't": "did not", "doesn't": "does not", "don't": "do not",
128     "hadn't": "had not", "hasn't": "has not", "haven't": "have not", "he'd": "he would",
129     "he'll": "he will", "he's": "he is", "how'd": "how did", "how's": "how is",
130     "i'd": "i would", "i'll": "i will", "i'm": "i am", "i've": "i have",
131     "isn't": "is not", "it'd": "it would", "it'll": "it will", "it's": "it is",
132     "let's": "let us", "ma'am": "madam", "mightn't": "might not", "mustn't": "must not",

```

```

133     "shan't": "shall not", "she'd": "she would", "she'll": "she will", "she's": "she is",
134     "shouldn't": "should not", "should've": "should have", "that's": "that is",
135     "there's": "there is", "they'd": "they would", "they'll": "they will",
136     "they're": "they are", "they've": "they have", "wasn't": "was not", "we'd": "we would",
137     "we'll": "we will", "we're": "we are", "we've": "we have", "weren't": "were not",
138     "what'll": "what will", "what're": "what are", "what's": "what is", "what've": "what
    have",
139     "where's": "where is", "who'll": "who will", "who's": "who is", "won't": "will not",
140     "wouldn't": "would not", "would've": "would have", "you'd": "you would",
141     "you'll": "you will", "you're": "you are", "you've": "you have"
142 }
143
144 def preprocess_sentence(sentence, remove_stopwords=True):
145     sentence = sentence.lower() # 소문자화
146     sentence = BeautifulSoup(sentence, "lxml").text # HTML 태그 제거
147     sentence = re.sub(r'\s*([^\s])\s*', ' ', sentence) # 괄호 내용 제거
148     sentence = re.sub("'", '', sentence) # 따옴표 제거
149     sentence = ' '.join([contractions[t] if t in contractions else t
150                          for t in sentence.split(" ")]) # 약어 정규화
151     sentence = re.sub(r"'s\b", "", sentence) # 소유격 제거
152     sentence = re.sub("[^a-zA-Z]", " ", sentence) # 영문 외 문자 제거
153     sentence = re.sub('[m]{2,}', 'mm', sentence) # mmm -> mm
154     if remove_stopwords: # 본문(Text)은 불용어 제거
155         tokens = ' '.join(w for w in sentence.split() if w not in stop_words and len(w) > 1)
156     else: # 요약(Summary)은 문장 형태 유지 위해 불용어 보존
157         tokens = ' '.join(w for w in sentence.split() if len(w) > 1)
158     return tokens
159
160 # 본문은 불용어 제거, 요약은 불용어 유지
161 clean_text = [preprocess_sentence(s, True) for s in data['Text']]
162 clean_summary = [preprocess_sentence(s, False) for s in data['Summary']]
163
164 data = pd.DataFrame({'text': clean_text, 'summary': clean_summary})
165 data.replace('', np.nan, inplace=True) # 정제 후 빈 문자열 -> 결측 처리
166 data.dropna(axis=0, inplace=True)
167 print("정제 후 샘플 수:", len(data))
168
169
170 # %%
171 # =====
172 # Cell 3. (전처리 - 길이 분석) 최대 길이 결정
173 # =====
174 text_len = [len(s.split()) for s in data['text']]
175 summary_len = [len(s.split()) for s in data['summary']]
176
177 print('본문 길이 평균/최대: %.1f / %d' % (np.mean(text_len), max(text_len)))
178 print('요약 길이 평균/최대: %.1f / %d' % (np.mean(summary_len), max(summary_len)))
179
180 plt.figure(figsize=(10, 4))
181 plt.subplot(1, 2, 1); plt.hist(text_len, bins=40); plt.title('Text length')
182 plt.subplot(1, 2, 2); plt.hist(summary_len, bins=40); plt.title('Summary length')
183 plt.tight_layout(); plt.show()
184
185 # 분포를 보고 결정 (Amazon Reviews 표준값)
186 text_max_len = 50
187 summary_max_len = 8
188
189 def below_threshold_len(max_len, lengths):
190     cnt = sum(1 for l in lengths if l <= max_len)
191     print('전체 중 %d 이하 비율: %.4f' % (max_len, cnt / len(lengths)))
192
193 below_threshold_len(text_max_len, text_len)
194 below_threshold_len(summary_max_len, summary_len)
195
196 # 최대 길이 이하 샘플만 사용
197 data = data[data['text'].apply(lambda x: len(x.split()) <= text_max_len)]
198 data = data[data['summary'].apply(lambda x: len(x.split()) <= summary_max_len)]

```

```

199 print("길이 필터 후 샘플 수:", len(data))
200
201
202 # %%
203 # =====
204 # Cell 4. 디코더용 시작/종료 토큰 부착
205 # =====
206 data['summary'] = data['summary'].apply(lambda x: 'sostoken ' + x + ' eostoken')
207 data.head()
208
209
210 # %%
211 # =====
212 # Cell 5. (전처리 4단계 - 데이터셋 분리)
213 # =====
214 encoder_input = np.array(data['text'])
215 decoder_input = np.array(data['summary'])
216
217 indices = np.arange(encoder_input.shape[0])
218 np.random.shuffle(indices)
219 encoder_input = encoder_input[indices]
220 decoder_input = decoder_input[indices]
221
222 n_test = int(len(encoder_input) * 0.2)
223 encoder_input_train, encoder_input_test = encoder_input[n_test:], encoder_input[:n_test]
224 decoder_train_raw, decoder_test_raw = decoder_input[n_test:], decoder_input[:n_test]
225 print('train:', len(encoder_input_train), ' / test:', len(encoder_input_test))
226
227
228 # %%
229 # =====
230 # Cell 6. (전처리 5단계 - 인코딩) 정수 인코딩 + 희귀단어 제거 + 패딩
231 # =====
232 # ---- 본문 토크나이저 ----
233 src_tokenizer = Tokenizer()
234 src_tokenizer.fit_on_texts(encoder_input_train)
235
236 threshold = 7
237 total_cnt = len(src_tokenizer.word_index)
238 rare_cnt = sum(1 for _, c in src_tokenizer.word_counts.items() if c < threshold)
239 src_vocab = total_cnt - rare_cnt + 1
240 print('본문 단어집합 크기:', src_vocab)
241
242 src_tokenizer = Tokenizer(num_words=src_vocab)
243 src_tokenizer.fit_on_texts(encoder_input_train)
244 encoder_input_train = src_tokenizer.texts_to_sequences(encoder_input_train)
245 encoder_input_test = src_tokenizer.texts_to_sequences(encoder_input_test)
246
247 # ---- 요약 토크나이저 ----
248 tar_tokenizer = Tokenizer()
249 tar_tokenizer.fit_on_texts(decoder_train_raw)
250
251 threshold = 6
252 total_cnt = len(tar_tokenizer.word_index)
253 rare_cnt = sum(1 for _, c in tar_tokenizer.word_counts.items() if c < threshold)
254 tar_vocab = total_cnt - rare_cnt + 1
255 print('요약 단어집합 크기:', tar_vocab)
256
257 tar_tokenizer = Tokenizer(num_words=tar_vocab)
258 tar_tokenizer.fit_on_texts(decoder_train_raw)
259 decoder_input_train = tar_tokenizer.texts_to_sequences(decoder_train_raw)
260 decoder_input_test = tar_tokenizer.texts_to_sequences(decoder_test_raw)
261
262 # ---- 요약이 사실상 비어있는(토큰 sos/eos만 남은) 샘플 제거 ----
263 def drop_empty(enc, dec):
264     keep = [i for i, s in enumerate(dec) if len(s) > 2] # sos,eos 외 단어 1개 이상
265     return [enc[i] for i in keep], [dec[i] for i in keep]

```

```

266
267 encoder_input_train, decoder_input_train = drop_empty(encoder_input_train,
    decoder_input_train)
268 encoder_input_test, decoder_input_test = drop_empty(encoder_input_test,
    decoder_input_test)
269 print('정제 후 train:', len(encoder_input_train), ' / test:', len(encoder_input_test))
270
271 # ---- 패딩 ----
272 encoder_input_train = pad_sequences(encoder_input_train, maxlen=text_max_len, padding='post')
273 encoder_input_test = pad_sequences(encoder_input_test, maxlen=text_max_len, padding='post')
274 decoder_padded_train = pad_sequences(decoder_input_train, maxlen=summary_max_len,
    padding='post')
275 decoder_padded_test = pad_sequences(decoder_input_test, maxlen=summary_max_len,
    padding='post')
276
277 # ---- teacher forcing: 입력(마지막 토큰 제외) / 정답(첫 토큰 제외) ----
278 decoder_input_train = decoder_padded_train[:, :-1]
279 decoder_target_train = decoder_padded_train[:, 1:]
280 decoder_input_test = decoder_padded_test[:, :-1]
281 decoder_target_test = decoder_padded_test[:, 1:]
282 print('shape:', encoder_input_train.shape, decoder_input_train.shape,
    decoder_target_train.shape)
283
284
285 # %%
286 # =====
287 # Cell 7. 모델 정의 (seq2seq + Attention)
288 # =====
289 embedding_dim = 128
290 hidden_size = 256
291
292 # ----- Encoder: 3-stacked LSTM -----
293 encoder_inputs = Input(shape=(text_max_len,))
294 enc_emb = Embedding(src_vocab, embedding_dim)(encoder_inputs)
295
296 encoder_lstm1 = LSTM(hidden_size, return_sequences=True, return_state=True,
    dropout=0.4, recurrent_dropout=0.4)
297 output1, h1, c1 = encoder_lstm1(enc_emb)
298
299
300 encoder_lstm2 = LSTM(hidden_size, return_sequences=True, return_state=True,
    dropout=0.4, recurrent_dropout=0.4)
301 output2, h2, c2 = encoder_lstm2(output1)
302
303
304 encoder_lstm3 = LSTM(hidden_size, return_sequences=True, return_state=True,
    dropout=0.4, recurrent_dropout=0.4)
305 encoder_outputs, state_h, state_c = encoder_lstm3(output2)
306
307
308 # ----- Decoder -----
309 decoder_inputs = Input(shape=(None,))
310 dec_emb_layer = Embedding(tar_vocab, embedding_dim)
311 dec_emb = dec_emb_layer(decoder_inputs)
312
313 decoder_lstm = LSTM(hidden_size, return_sequences=True, return_state=True,
    dropout=0.4, recurrent_dropout=0.2)
314 decoder_outputs, _, _ = decoder_lstm(dec_emb, initial_state=[state_h, state_c])
315
316
317 # ----- Attention -----
318 attn_layer = AttentionLayer(name='attention_layer')
319 attn_out, attn_states = attn_layer([encoder_outputs, decoder_outputs])
320 decoder_concat_input = Concatenate(axis=-1, name='concat_layer')([decoder_outputs, attn_out])
321
322 decoder_softmax_layer = Dense(tar_vocab, activation='softmax')
323 decoder_softmax_outputs = decoder_softmax_layer(decoder_concat_input)
324
325 model = Model([encoder_inputs, decoder_inputs], decoder_softmax_outputs)
326 model.compile(optimizer='rmsprop', loss='sparse_categorical_crossentropy')
327 model.summary()

```

```

328
329
330 # %%
331 # =====
332 # Cell 8. (루브릭2) 학습 - EarlyStopping
333 # =====
334 es = EarlyStopping(monitor='val_loss', mode='min', verbose=1, patience=2)
335
336 history = model.fit(
337     x=[encoder_input_train, decoder_input_train],
338     y=decoder_target_train,
339     validation_data=([encoder_input_test, decoder_input_test], decoder_target_test),
340     batch_size=256, epochs=50, callbacks=[es]
341 )
342
343
344 # %%
345 # =====
346 # Cell 9. (루브릭2) train / validation loss 그래프
347 # =====
348 plt.figure(figsize=(7, 4))
349 plt.plot(history.history['loss'], label='train_loss')
350 plt.plot(history.history['val_loss'], label='val_loss')
351 plt.title('Training / Validation Loss')
352 plt.xlabel('epoch'); plt.ylabel('loss'); plt.legend(); plt.show()
353
354
355 # %%
356 # =====
357 # Cell 10. 추론용 인코더/디코더 모델 구성
358 # =====
359 src_index_to_word = src_tokenizer.index_word
360 tar_word_to_index = tar_tokenizer.word_index
361 tar_index_to_word = tar_tokenizer.index_word
362
363 # Encoder inference
364 encoder_model = Model(inputs=encoder_inputs, outputs=[encoder_outputs, state_h, state_c])
365
366 # Decoder inference
367 decoder_state_input_h = Input(shape=(hidden_size,))
368 decoder_state_input_c = Input(shape=(hidden_size,))
369 decoder_hidden_state_input = Input(shape=(text_max_len, hidden_size))
370
371 dec_emb2 = dec_emb_layer(decoder_inputs)
372 decoder_outputs2, state_h2, state_c2 = decoder_lstm(
373     dec_emb2, initial_state=[decoder_state_input_h, decoder_state_input_c])
374
375 attn_out_inf, attn_states_inf = attn_layer([decoder_hidden_state_input, decoder_outputs2])
376 decoder_inf_concat = Concatenate(axis=-1)([decoder_outputs2, attn_out_inf])
377 decoder_outputs2 = decoder_softmax_layer(decoder_inf_concat)
378
379 decoder_model = Model(
380     [decoder_inputs, decoder_hidden_state_input, decoder_state_input_h,
381      decoder_state_input_c],
382     [decoder_outputs2, state_h2, state_c2])
383
384 def decode_sequence(input_seq):
385     e_out, e_h, e_c = encoder_model.predict(input_seq, verbose=0)
386     target_seq = np.zeros((1, 1))
387     target_seq[0, 0] = tar_word_to_index['sostoken']
388
389     stop = False
390     decoded = ''
391     while not stop:
392         output_tokens, h, c = decoder_model.predict([target_seq, e_out, e_h, e_c], verbose=0)
393         idx = np.argmax(output_tokens[0, -1, :])

```

```

394     word = tar_index_to_word.get(idx, '')
395     if word != 'eostoken' and word != '':
396         decoded += ' ' + word
397     if word == 'eostoken' or len(decoded.split()) >= (summary_max_len - 1):
398         stop = True
399     target_seq = np.zeros((1, 1)); target_seq[0, 0] = idx
400     e_h, e_c = h, c
401     return decoded.strip()
402
403
404 def seq2text(seq):
405     return ' '.join(src_index_to_word[i] for i in seq if i != 0)
406
407 def seq2summary(seq):
408     return ' '.join(tar_index_to_word[i] for i in seq
409                     if i != 0
410                     and i != tar_word_to_index['sostoken']
411                     and i != tar_word_to_index['eostoken'])
412
413
414 # %%
415 # =====
416 # Cell 11. (루브릭2) Abstractive 결과 확인 - 핵심단어 포함 여부 점검
417 # =====
418 for i in range(5):
419     print("원문      :", seq2text(encoder_input_test[i]))
420     print("실제 요약 :", seq2summary(decoder_padded_test[i]))
421     print("예측 요약 :", decode_sequence(encoder_input_test[i].reshape(1, text_max_len)))
422     print("-" * 60)
423
424
425 # %%
426 # =====
427 # Cell 12. (루브릭1) Extractive 요약 - TextRank (summa)
428 # - summa 는 여러 문장으로 된 '긴' 원문에서 잘 동작 (Amazon 리뷰는 짧으므로
429 #   원본 Text 중 긴 샘플로 시연). 뉴스 데이터셋이면 더 자연스럽게 동작.
430 # =====
431 # !pip install summa
432 from summa.summarizer import summarize
433
434 raw = pd.read_csv("Reviews.csv", nrows=100000)[['Text', 'Summary']].dropna()
435 long_reviews = raw[raw['Text'].apply(lambda x: len(x.split('.')) >=
436                                     5)].reset_index(drop=True)
437
438 for i in range(3):
439     text = long_reviews['Text'][i]
440     print("[원문]\n", text[:500], "... \n")
441     print("[Extractive(TextRank)]\n", summarize(text, ratio=0.3), "\n")
442     print("[Abstractive(seq2seq)]\n",
443           decode_sequence(
444               pad_sequences(src_tokenizer.texts_to_sequences([preprocess_sentence(text)]),
445                           maxlen=text_max_len, padding='post')), "\n")
446     print("=" * 70)
447
448 # %%
449 # =====
450 # Cell 13. (루브릭1) 비교 결과 표로 정리
451 # - 아래 표의 '문법 완성도', '핵심단어 포함' 칸은 본인 실제 출력으로 채울 것
452 # =====
453 compare = pd.DataFrame({
454     '비교 항목': ['문법 완성도', '핵심단어 포함', '요약 길이/압축', '새로운 표현 생성'],
455     'Extractive (TextRank)': [
456         '원문 문장을 그대로 추출 → 항상 문법적으로 완전',
457         '문장 통째로 추출 → 핵심어 누락 거의 없음(불필요 정보도 포함)',
458         '문장 단위라 상대적으로 길다',
459         '불가능 (원문 문장만 선택)']

```

```
460     ],
461     'Abstractive (seq2seq+Attn)': [
462         '단어를 새로 생성 → 짧으면 자연스럽지만 반복/비문 가능',
463         '핵심 명사 위주 압축 → 학습 잘되면 포함, 희귀어/OOV는 누락 가능',
464         '매우 짧게 압축 (몇 단어)',
465         '가능 (원문에 없는 단어 생성)'
466     ],
467 })
468 pd.set_option('display.max_colwidth', None)
469 print(compare.to_string(index=False))
470
```